CS300 – Sabancı University Homework 1

1. Introduction

In this homework, you will implement a Music Playlist Manager. The playlist must be maintained using a circular doubly linked list structure, where each song is stored as a node.

Your program will process a sequence of commands to dynamically manage the playlist, including adding/removing songs and navigating through the playlist. The implementation must preserve the circular doubly linked list structure throughout all operations.

2. Implementation

Your program must support all commands described in the next section, and the playlist must preserve its circular doubly linked list structure after every operation.

You **must not** use any STL containers such as vector, list, deque, map, set, queue, or stack. The only allowed libraries are <iostream>, <string>, and <sstream>. Therefore, all required data structures must be implemented manually using **pointers**.

You will submit only one file named main.cpp. Your program must compile using C++17, read input from standard input, and write output to standard output. Do not print any prompts such as *Enter command:* or any extra text that is not explicitly specified.

Data Structure Requirements

Each node in the playlist must store the following fields:

- songId
- songName
- artistName
- duration
- playCount
- next
- prev

The playlist itself must maintain head, tail, currentSong, and playlistSize.

At all times, the list must satisfy the circular doubly linked list properties:

- head->prev == tail
- tail->next == head

3. Interaction with the User

Your program reads commands from standard input and prints the required messages to standard output. Commands are separated by whitespace and the program terminates only when EXIT is received. songName and artistName are given as single tokens (no spaces).

ADD

If *songId* is unique, the song is appended to the playlist and the program prints [Song added.]. Otherwise, no song is added and the program prints [Song already exists.].

REMOVE

If the playlist is empty, print [Playlist is empty.]. If a song with songId exists, remove it and print [Song removed.]. Otherwise, print [Not found.]. If the removed song is the currently playing song, the current song becomes the next song in the list.

PLAY

If the playlist is empty, print Playlist is empty. If songId exists, perform the relevant operations and print [Playing: <songName>]. Otherwise, print Not found.

NEXT

If there is no currently playing song, print [No song is currently playing.]. Otherwise print Playing: <songName>.

PREV

Perform the relevant operations and move to previous song.

MOVE_AFTER (Foramt : MOVE_AFTER A B)

If either A or B does not exist, print Not found. If the move is ignored (for example A == B, the playlist size is less than 2, or A is already immediately after B), print [Move ignored.] Otherwise, perform the relevant operations and print [Song moved.].

REVERSE_SEGMENT (Format : REVERSE_SEGMENT L R)

Positions are 1-indexed and inclusive. If the range is invalid (L < 1, R > playlistSize, or L >= R), print [Invalid range.]. Otherwise, Reverse the songs between positions L and R (1-indexed) and print [Segment reversed.].

JUMP (Format : JUMP K)

If K is negative, print Invalid jump. Otherwise, Perform the relevant operations and print [Playing: <songName>.].

PRINT

Output playlist starting from head.

EXIT

Print [Exiting. Goodbye!] and terminate the program.

If an unknown command is given, the program prints [Invalid command.].

Important Notes

- Your program must read from standard input and print to standard output.
- Do not print any menu, prompt, or extra text.
- Print only the exact outputs described above.
- You must submit only one file named main.cpp. The submitted code must be a complete C++ program and must contain exactly one main() function.

4. Sample Runs

The following sample runs are provided to illustrate the expected behavior of the program and the exact output format. In total, there are 10 sample runs. Only 6 are given here, the other 4 are hidden sample runs. In the following examples, **bold** lines represent the input entered by the user, and *italic* lines represent the expected output of the program.

4.1 Sample Run 1

ADD 1 Believer ImagineDragons 204
Song added.
ADD 2 Yellow Coldplay 269
Song added.
ADD 3 Halo Beyonce 261
Song added.
REMOVE 2
Song removed.
PRINT
Playlist: Believer Halo
Current: None
EXIT
Exiting. Goodbye!

4.2 Sample Run 2

ADD 10 Numb LinkinPark 185
Song added.
ADD 20 FixYou Coldplay 295
Song added.
ADD 30 Skyfall Adele 286
Song added.

PLAY 30
Playing: Skyfall
NEXT
Playing: Numb
PRINT
Playlist: Numb FixYou Skyfall
Current: Numb
PREV
Playing: Skyfall
PREV
Playing: FixYou
PRINT
Playlist: Numb FixYou Skyfall
Current: FixYou
EXIT
Exiting. Goodbye!

4.3 Sample Run 3

ADD 101 ShapeOfYou EdSheeran 233
Song added.
ADD 102 Levitating DuaLipa 203
Song added.
ADD 103 BadGuy BillieEilish 194
Song added.
MOVE_AFTER 102 103
Song moved.
PRINT
Playlist: ShapeOfYou BadGuy Levitating
Current: None
MOVE_AFTER 102 102
Move ignored.
PRINT
Playlist: ShapeOfYou BadGuy Levitating
Current: None
EXIT
Exiting. Goodbye!

4.4 Sample Run 4

ADD 201 Thunder ImagineDragons 187
Song added.
ADD 202 VivaLaVida Coldplay 242
Song added.
ADD 203 RollingInTheDeep Adele 228
Song added.
ADD 204 BlindingLights TheWeeknd 200
Song added.
REMOVE 201
Song removed.
REMOVE 204
Song removed.
PRINT
Playlist: VivaLaVida RollingInTheDeep
Current: None
NEXT
No song is currently playing.
NEXT
No song is currently playing.
PRINT
Playlist: VivaLaVida RollingInTheDeep
Current: None
EXIT
Exiting. Goodbye!

4.5 Sample Run 5

PRINT
Playlist:
Current: None
ADD 301 Diamonds Rihanna 225
Song added.
ADD 302 LoseYourself Eminem 326
Song added.
PLAY 999
Not found.
PLAY 302
Playing: LoseYourself

JUMP 1
Playing: Diamonds
JUMP 3
Playing: LoseYourself
PRINT
Playlist: Diamonds LoseYourself
Current: LoseYourself
EXIT
Exiting. Goodbye!

4.6 Sample Run 6

ADD 801 Havana CamilaCabello 217
Song added.
ADD 802 Senorita ShawnMendes 191
Song added.
ADD 803 Toxic BritneySpears 198
Song added.
ADD 804 SomeoneLikeYou Adele 285
Song added.
PLAY 804
Playing: SomeoneLikeYou
REVERSE_SEGMENT 2 4
Segment reversed.
PRINT
Playlist: Havana SomeoneLikeYou Toxic Senorita
Current: SomeoneLikeYou
EXIT
Exiting. Goodbye!

Important: Your program's input/output format must exactly match the structure described in this document and illustrated in the sample runs. Any difference in the printed output, including missing lines, extra lines, or mismatched text, may cause the automatic grading system to assign 0 points for that test case.

5. Submission

You must submit your homework through SUCourse. Submit **only** your main.cpp by pasting its full content into the **Answer Box,** and do not upload any additional files. Paste only C++ code (do not include extra text or Markdown fences such as ```). Your program will be compiled using C++17 and executed automatically by the grading system.

If you have any questions, please contact me at : erfan.rezaei@sabanciuniv.edu